

4-Bit to 2-Bit Priority Encoder — Theory of Operation

Functional Description

A 4-bit to 2-bit priority encoder is a combinational circuit that accepts four input lines, (I_3 , I_2 , I_1 , I_0), and produces a 2-bit encoded output (Y_1Y_0). The priority order in this implementation is ($I_3 > I_2 > I_1 > I_0$), so a higher-priority active input masks all lower-priority inputs.

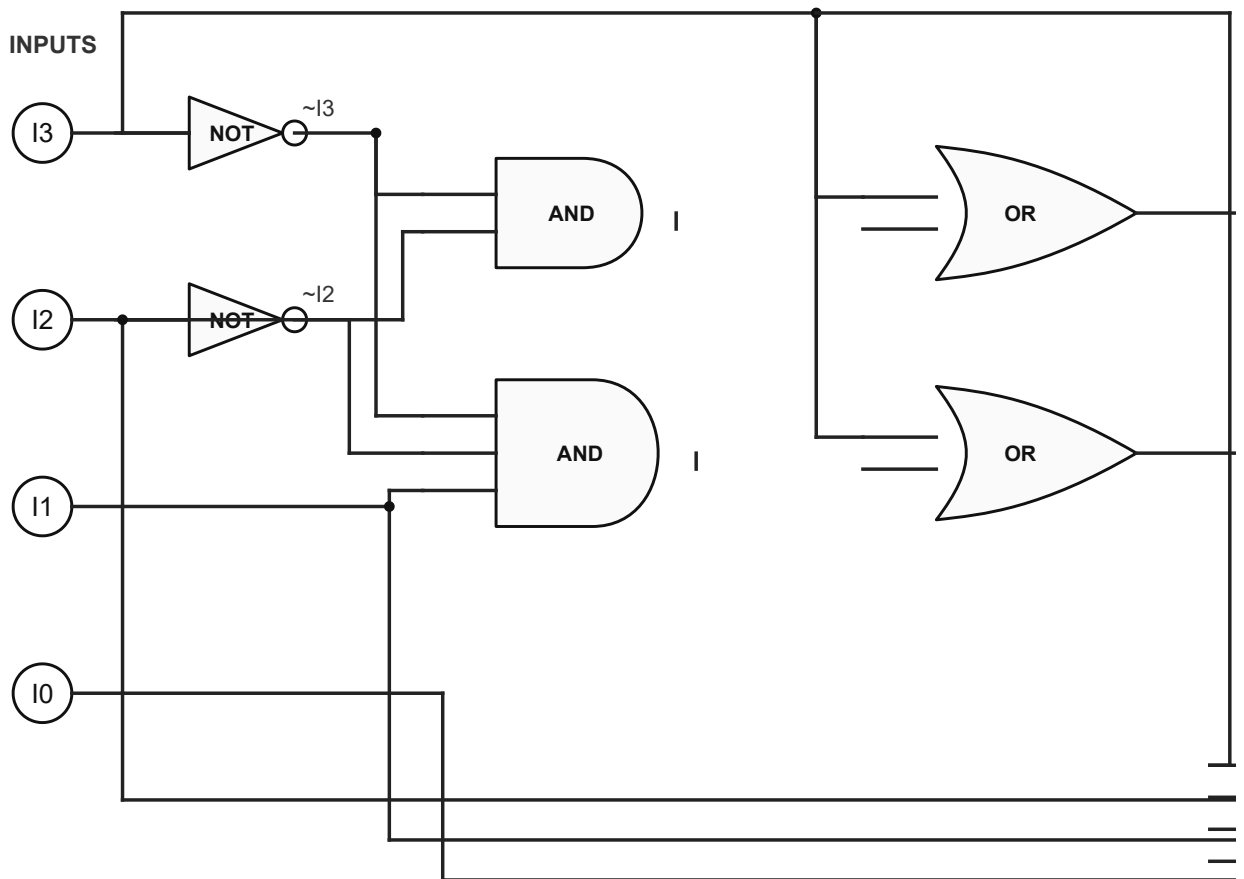
The `valid` output indicates whether at least one input is active. When (I_3) is active, the encoded output is `11`; when ($I_3=0$) and ($I_2=1$), it is `10`; when only the lower-priority conditions are satisfied, the encoder produces the corresponding lower code. With all inputs low, `valid=0` and (Y_1Y_0) is treated as don't-care by the priority specification.

The circuit diagram below represents the explicit gate-level structure used by the RTL: two NOT gates generate the priority masks, AND gates form the masked lower-priority terms, OR gates form (Y_1) and (Y_0), and a 4-input OR generates `valid`. This is an RTL-derived structural representation; synthesis may legally simplify equivalent Boolean expressions.

Circuit Diagram

4-Bit to 2-Bit Priority Encoder — RTL-Derived Gate-Level Schematic

Priority: $I_3 > I_2 > I_1 > I_0$



Gate symbols follow ANSI-style digital logic conventions; schematic mirrors the explicit priority-masking structure represented

Figure: RTL-derived gate-level schematic using standard ANSI-style NOT, AND, and OR symbols.

How It Works, Step by Step

1. **Generate the priority masks.** The first NOT gate produces $(\overline{I_3})$. The second produces $(\overline{I_2})$. These inverted signals prevent lower-priority inputs from contributing when a higher-priority input is active.
2. **Form the (I_2) term.** The 2-input AND gate combines $(\overline{I_3})$ and (I_2) . Its output is therefore (1) only when (I_2) is active while the higher-priority input (I_3) is inactive: $[\overline{I_3} \cdot I_2]$
3. **Form the (I_1) term.** The 3-input AND gate combines $(\overline{I_3})$, $(\overline{I_2})$, and (I_1) . It becomes (1) only when both higher-priority inputs are inactive and (I_1) is active: $[\overline{I_3} \cdot \overline{I_2} \cdot I_1]$
4. **Generate (Y_1) .** The upper OR gate combines (I_3) with the masked (I_2) term: $[Y_1 = I_3 + (\overline{I_3} \cdot I_2)]$ Thus (Y_1) is asserted for either the highest-priority input (I_3) or the next-priority input (I_2) when (I_3) is inactive.
5. **Generate (Y_0) .** The lower OR gate combines (I_3) with the masked (I_1) term: $[Y_0 = I_3 + (\overline{I_3} \cdot \overline{I_2} \cdot I_1)]$ Therefore (Y_0) is high for (I_3) , or for (I_1) when both higher-priority inputs are low.
6. **Generate `valid`.** The 4-input OR receives (I_3) , (I_2) , (I_1) , and (I_0) : $[valid = I_3 + I_2 + I_1 + I_0]$ Consequently, `valid` becomes `1` whenever any request is present.
7. **Priority behavior.** For example, with $(I_3I_2I_1I_0=1011)$, $(\overline{I_3}=0)$, which disables both lower-priority AND paths. The two output OR gates therefore receive $(I_3=1)$, giving $(Y_1Y_0=11)$, while `valid=1`.